

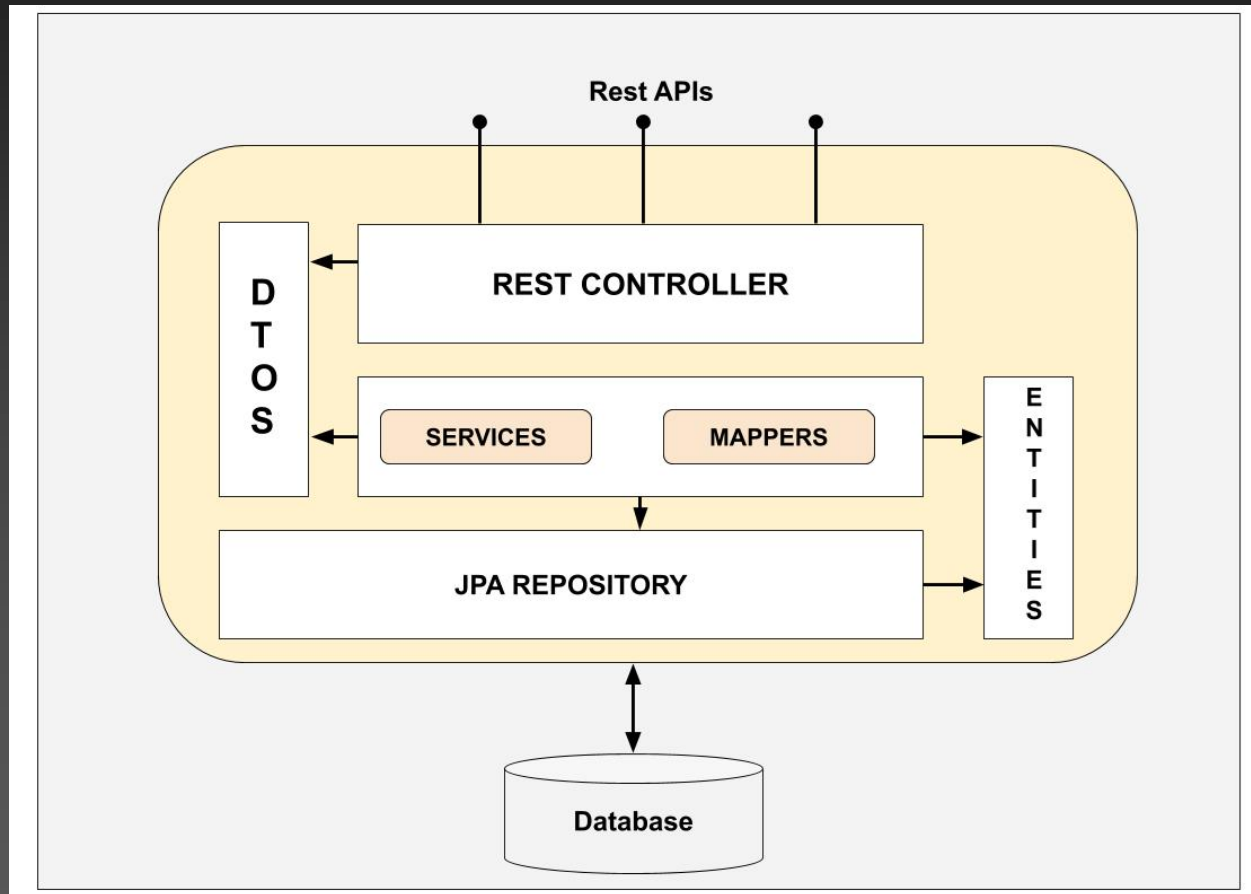
Spring Boot

Pattern DTO

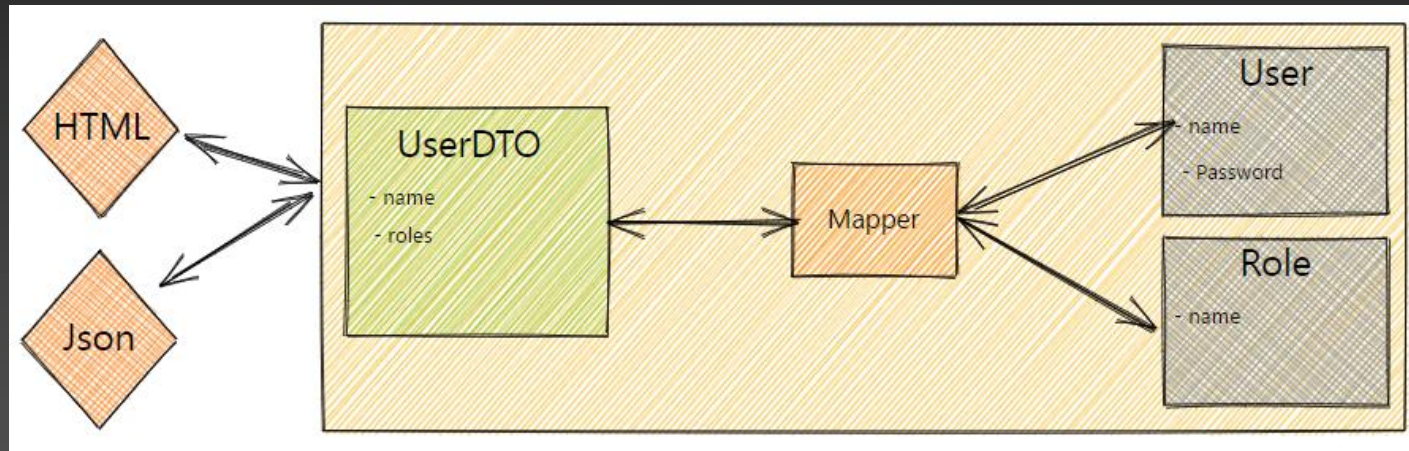
Pattern DTO

- I DTO o Data Transfer Object sono degli oggetti POJO che incapsulano dati e non contengono logica aziendale.
- Tali oggetti vengono inviati tra i componenti dell'applicazione.
- Lo scopo è di ridurre il numero di chiamate tra componenti.(es: tra client e server, server e server). Raggruppando più parametri all'interno di una singola chiamata.
- Vantaggi:
 - Disaccoppiamento delle classi entity dal layer controller.
 - Risposta ottimizzata in base alle esigenze del client.
 - Sicurezza.
 - Non più metodi che prendono tanti parametri in input.
- Svantaggi:
 - La conversione da entity a DTO e viceversa può essere «costoso».

Pattern DTO



Pattern DTO



Pattern DTO



```
@Entity
public class Utente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    private String email;
    private String password;

    @ManyToMany
    @JoinTable(name = "ruolo_utente",
        joinColumns = @JoinColumn(name = "id_utente", nullable = false),
        inverseJoinColumns = @JoinColumn(name = "id_ruolo", nullable = false))
    private List<Ruolo> ruoli;

}
```

```
@Entity
public class Ruolo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    private String nome;

}
```

```
public class LoginDTO {
    private String username;
    private String password;

    //getter - setter - costruttore

}
```

```
public class UtenteDTO {

    private String nome;
    private List<String> ruoli;

    //getter - setter - costruttori

}
```



Pattern DTO



```
@Component
public class UtenteMapper {

    public UtenteDTO toDTO(Utente u){
        String nome=u.getNome();
        List<String> ruoli=
u.getRuoli().stream().map(Ruolo::getNome).toList();
        return new UtenteDTO(nome,ruoli);
    }
}
```



Pattern DTO



```
@PostMapping("/utente/login")
public ResponseEntity<UtenteDTO> login(@RequestBody LoginDTO request){
    return ResponseEntity.status(HttpStatus.OK).body(service.login(request.getUsername(),request.getPassword()));
}
```

```
@Service
public class UtenteServiceImpl implements UtenteService{

    private final UtenteRepository repo;
    private final UtenteMapper mapper;

    public UtenteServiceImpl(UtenteRepository repo, UtenteMapper mapper) {
        this.repo = repo;
        this.mapper = mapper;
    }

    @Override
    public UtenteDTO login(String email, String password){
        Utente u=repo.findByEmailAndPassword(email,password).orElseThrow(UtenteNonTrovatoException::new);
        return mapper.toDTO(u);
    }
}
```



Pattern DTO



HTTP request body

```
1  {  
2  |  ...."username":"admin@email.com",  
3  |  ...."password":"root"  
4  |  
5  }
```

HTTP response body

```
1  {  
2  |  
3  |  "nome": "admin",  
4  |  "ruoli": [  
5  |  |  "admin"  
6  |  ]  
7  }
```



Pattern DTO

